

CipherSafe - Four-Way Handshake Process

Alex Biddle

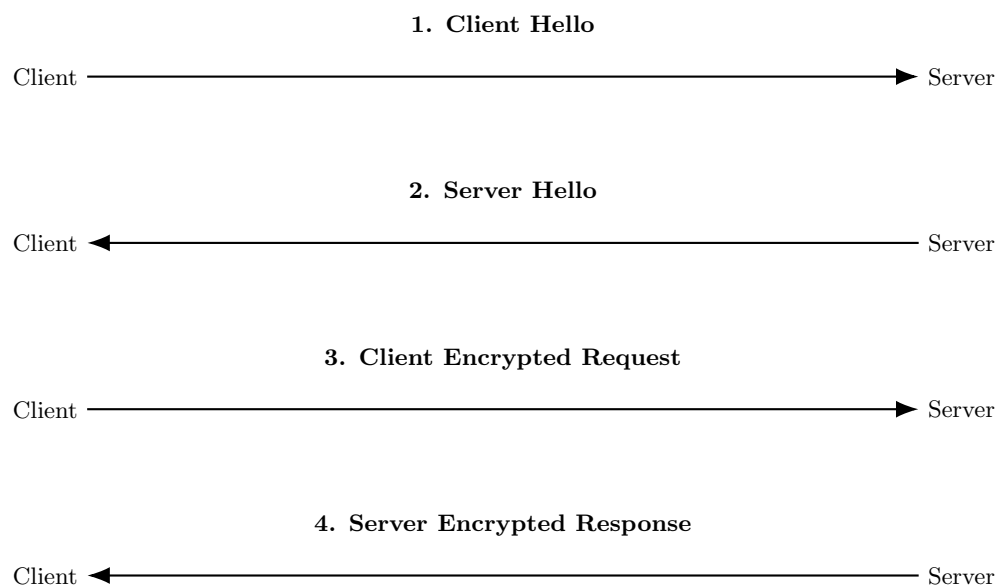
June 8, 2026

Four-Way Handshake

CipherSafe uses a four-way handshake protocol to authenticate the client, verify message integrity, exchange cryptographic keys, and establish an encrypted session for secure data transmission. The handshake operates entirely over **TLS 1.3** and **HTTPS 3.0** and guarantees confidentiality, integrity, and availability for all communications between the client and the backend Python server.

Four-way Handshake Objectives

- Prevent man-in-the-middle, replay, and key-substitution attacks.
- Bind client and server messages using cryptographic checksums.
- Protect all transmitted authentication and vault data using encryption.



Four-Way Handshake JSON Packet Details

Step 1: Client → Server (Client Hello)

JSON Field	Type / Format
protocol_version	string ("CipherSafe Handshake v1")
request_type	string
username	string (max 64 characters)
client_rsa_public_key	base64 (PEM-encoded RSA-2048 key)
timestamp	string (ISO8601Z, e.g., 2025-11-10T15:30:00Z)
client_checksum	base64url (SHA-256(username, timestamp))

Step 2: Server → Client (Server Hello)

JSON Field	Type / Format
protocol_version	string ("CipherSafe Handshake v1")
response_status	string
username	string (verified username)
timestamp	string (ISO8601Z)
server_checksum	base64url (SHA-256(username, timestamp))
server_aes_session_key	base64url (RSA-OAEP(AES-256 key))
server_rsa_public_key	base64 (PEM-encoded RSA public key)
server_key_identifier	UUID ("550e8400-e29b-41d4-a716-446655440000")
server_key_expiry	string (ISO8601Z UTC expiration)
server_digital_signature	base64url (RSA-PSS-SHA256(session_key + key_id + timestamp))
server_ciphertext	base64url (RSA-OAEP-SHA256(client_salt))

Step 3: Client → Server (Encrypted Request)

JSON Field	Type / Format
protocol_version	string ("CipherSafe Handshake v1")
request_type	string
username	string
timestamp	string (ISO8601Z)
client_checksum	base64url (SHA-256(ciphertext AES_session_key))
server_aes_session_key	base64url (AES-256 key for current session)
server_key_identifier	UUID
client_ciphertext	base64url (AES-GCM(nonce AAD payload))

Step 4: Server → Client (Encrypted Response)

JSON Field	Type / Format
protocol_version	string ("CipherSafe Handshake v1")
response_status	string
username	string (verified username)
timestamp	string (ISO8601Z)
server_checksum	base64url (SHA-256(encrypted_data AES_session_key))
server_ciphertext	base64url (AES-GCM(nonce AAD payload))
server_digital_signature	base64url (RSA-PSS-SHA256(checksum response_status))

Client Request and Server Response Tags

HTTP Interfaces

Endpoint (HTTPS)	Purpose
POST /api/handshake/hello	Client Hello / Server Hello
POST /api/handshake/secure	Encrypted Request / Encrypted Response
POST /api/logout	Client logout

Client Request Types

Client request_type
login
signup
vault_fetch_accounts
vault_fetch_account_password
vault_add_account
vault_update_account
vault_delete_account
vault_update_master_password
logout

Server Response Types

Server response_status
success
invalid_account
authentication_failed
user_account_exists
invalid_checksum
failure

Encryption Process between Client and Server

Step 1: Client Hello

In the first phase, the client initiates the handshake by transmitting its RSA-2048 public key, username, current ISO 8601Z timestamp, and a checksum computed as `SHA_256(username + server_timestamp)`. No encryption is used at this stage, since the goal is to allow the server to authenticate the message format and obtain the client's RSA public key. This public key will later be used by the server to securely wrap an AES-256 session key. The checksum provides message integrity and prevents simple forgery or replay of unauthenticated Client Hello packets.

Step 2: Server Hello

Upon receiving the Client Hello, the server generates a new 32-byte AES-256 session key and encrypts this key using the client's RSA public key under RSA-OAEP with SHA-256 as the padding hash. The wrapped AES key is included in the Server Hello message together with the server's own RSA public key, its key identifier, and key expiry metadata. The server additionally computes a checksum `SHA_256(username + server_timestamp)` and produces a digital signature using RSA-PSS with SHA-256 over the concatenation (`session_key + key_id + server_timestamp`). This step ensures that only the legitimate client—is in possession of the private half of its RSA key—can decrypt the AES session key and establish the symmetric context used in subsequent encrypted communication.

Step 3: Client Encrypted Request

After successfully decrypting the AES-256 session key with its private RSA key, the client switches to symmetric authenticated encryption. All subsequent payloads are encrypted with AES-256 in Galois/Counter Mode (GCM) using a freshly generated 12-byte nonce for each request. The client transmits its encrypted application data as `client_ciphertext = AES-GCM(nonce + ciphertext + tag)`, along with an integrity checksum `SHA_256(client_ciphertext + AES_key)`. The presence of a valid checksum and the correct AES key proves to the server that the client properly derived and is using the shared symmetric key from Step 2. From this point forward, RSA is no longer used for encrypting payload data.

Step 4: Server Encrypted Response

Once the server decrypts and processes the client's AES-GCM message, it constructs an encrypted response using the same AES-256 session key. The response payload is encrypted as `server_ciphertext = AES-GCM(nonce + ciphertext + tag)`, and a checksum `SHA_256(server_ciphertext + AES_key)` is calculated to guarantee message integrity. To authenticate the origin of the response, the server signs the concatenation of (`server_checksum + response_status`) with its RSA private key using RSA-PSS and SHA-256. This final message provides both confidentiality through AES-GCM and authenticity through the server's RSA signature, completing the four-way handshake and establishing a fully trusted encrypted channel.

Protocol Cryptographic Properties

User Password Hashing Derivation

All user passwords are processed using a **two-stage Argon2id hashing pipeline** to prevent credential reuse and protect both client and server environments. The client first computes a **client_hashed_password** = Argon2id(password, client_salt), where the client_salt is randomly generated and stored by the server.

When the user logs in, the server provides the corresponding client_salt. The client re-computes the client hash locally and sends it to the server. The server then performs a second derivation: **server_hashed_password** = Argon2id(client_hashed_password, server_salt).

Only the final **server_hashed_password**, along with the **client_salt** and **server_salt**, is stored in the database. No plaintext password or intermediate hash ever leaves the client device unprotected.

Message Integrity and Replay Prevention

Each handshake message includes a **SHA-256 checksum** computed over all serialized message fields. This checksum ensures tamper detection and integrity validation prior to any decryption or signature verification.

Timestamps are validated within a strict **±5-second drift window** and verified against the current UTC clock to prevent message reuse. Nonces are accepted only once per session and tracked in-memory by the **ServerSessionHandler**, ensuring strict replay prevention.

Session expiration occurs automatically after 15 minutes. Expired sessions are securely destroyed, and standardized logout packets are generated by a dedicated cleanup thread running every 60 seconds.

Authentication and Verification

Mutual authentication is achieved through **RSA-2048 digital signatures** and **key fingerprint validation**. The server signs all outgoing responses using its **RSA-PSS (SHA-256)** private key. Clients verify these signatures using the corresponding **server RSA public key** distributed during the Server Hello phase.

Session key wrapping uses **RSA-OAEP (SHA-256)** to encrypt the AES-256 session key exchanged during the handshake. All server keypairs are automatically rotated every 24 hours; the private key is securely stored on disk with AES encryption and restricted file permissions, while the public key is derived dynamically from the private key and cached for active sessions.

Encryption and Confidentiality

All sensitive content—including handshake payloads, authentication data, and vault entries—is protected using **AES-256-GCM** authenticated encryption. Each ciphertext includes a 12-byte random nonce and a 16-byte authentication tag.

Associated data (AAD) binds contextual metadata such as timestamps and usernames to the encryption process, ensuring message authenticity. Decryption is only performed after both the checksum and RSA signature validations succeed, preventing ciphertext substitution or forgery.

Credential and Vault Protection

User credentials and vault contents are never stored or transmitted in plaintext. Vault entries are encrypted individually as AES-GCM ciphertext objects, containing encrypted account labels, usernames, URLs, and passwords.

When sent between client and server, each encrypted object includes a **server_checksum** or **client_checksum** field, generated as $\text{SHA-256}(\text{data} \parallel \text{AES_session_key})$. This guarantees detection of any tampering or transmission corruption prior to decryption.

Decryption occurs exclusively on the client using session-scoped AES keys derived during the handshake, ensuring true end-to-end confidentiality.

Overall Security Guarantee

The CipherSafe Four-Way Handshake operates entirely under **TLS 1.3**, providing layered encryption and mutual authentication across both the transport and application layers.

By combining **Argon2id**, **AES-256-GCM**, **RSA-OAEP**, **RSA-PSS**, and **SHA-256**, CipherSafe ensures confidentiality, integrity, and forward secrecy across all communication phases.

Strict timestamp validation, nonce enforcement, and 15-minute session TTL guarantee that every message is both unique and ephemeral, protecting against replay, forgery, and key-reuse attacks.

Cryptographic Algorithms:

- **RSA-2048(OAEP + SHA-256)**: Used for session key wrapping and secure transport of AES keys.
- **RSA-2048 (PSS + SHA-256)** – Used for digital signatures on all server responses.
- **AES-256-GCM** – Provides authenticated encryption for payloads and vault data.
- **Argon2id** – Derives secure client and server password hashes with unique salts.
- **SHA-256** – Generates message integrity checksums and detects tampering.